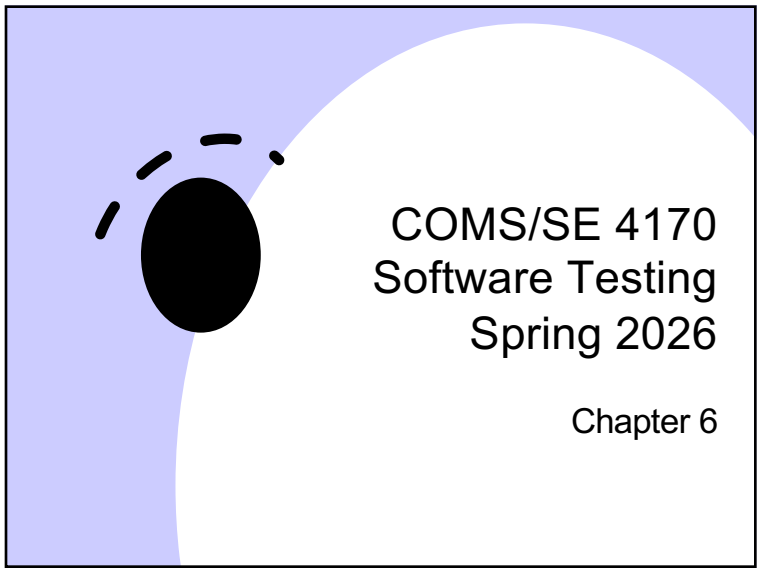
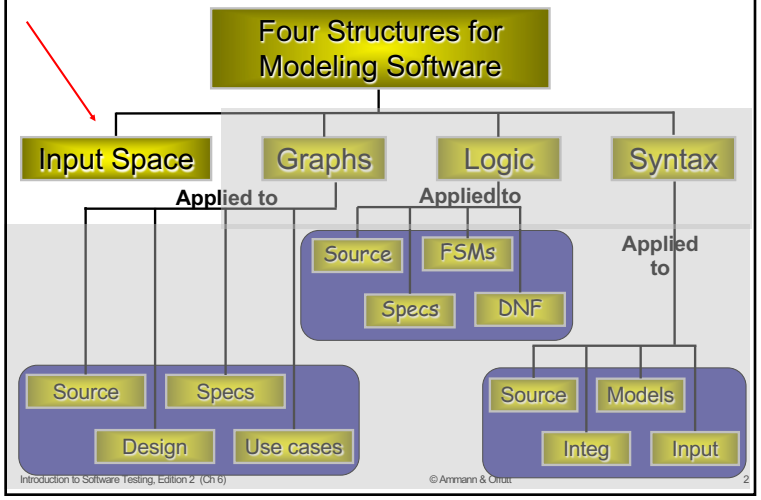


COMS/SE 4170 Software Testing Spring 2026

Chapter 6

1

Ch. 6 : Input Space Coverage



2

Input Space Partitioning

- Idea is that we can logically divide the input space into logical partitions of the input
- Independent of the RIPR model – focuses only on the input domain
- Input parameters:
  - Method parameters and non-local variables (unit testing), objects (integration testing), user inputs (system testing)

3

Other Terms

- Specification testing

4

Input Domains

- The input domain for a program contains all the possible inputs to that program
- For even small programs, the input domain is so large that it might as well be infinite
- Testing is fundamentally about choosing finite sets of values from the input domain
- *Input parameters* define the scope of the input domain
  - Parameters to a method
  - Data read from a file
  - Global variables
  - User level inputs
- Input domains are partitioned into regions (blocks)
- At least one value is chosen from each block

5

Foundations

- Equivalence classes
  - All inputs in a class are expected to behave in the same way

- *Domain D*
- *Partition scheme q* of *D*
- The partition *q* defines a *set of blocks*, $Bq = b_1, b_2, \ldots, b_Q$

6

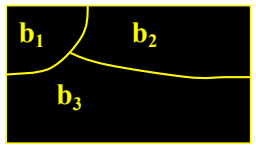
Foundations

- The partition must satisfy two properties:
  1. Blocks must be *pairwise disjoint* (no overlap)

$$b_i \cap b_j = \Phi, \forall i \neq j, b_i, b_j \in B_q$$

  2. Together the blocks *cover* the domain *D* (complete)

$$\bigcup_{b \in Bq} b = D$$



7

Using Partitions – Assumptions

- Choose a value from each block
- Each value is assumed to be equally useful for testing
- Application to testing
  - Find characteristics in the inputs : parameters, semantic descriptions, …
  - Partition each characteristic
  - Choose tests by combining values from characteristics
- Example Characteristics
  - Input X is null
  - Order of the input file F (sorted, inverse sorted, arbitrary, …)
  - Min separation of two aircraft
  - Input device (DVD, CD, VCR, computer, …)

8

Two Properties

- Blocks must not overlap (disjoint)
- Blocks cover the full input space (complete)

9

Example

- *Order of File F*
  - b_1= sorted ascending
  - b_2 = sorted descending
  - b_3 = arbitrary

- Not disjoint
  - The length 0 or 1 file belongs in all three

10

Valid Solution

- Keep characteristics limited to a single property (i.e. sorting and order)
  - *File F = sorted ascending*
    - b1= True
    - b2 – False
  - *File F sorted descending*
    - b1 = True
    - b2 = False

- Files of length 0 or 1 are in True category for both characteristics

11

Two (separate) Parts to Modeling

1. Choose characteristics and partitions
2. Combining tests (coverage criteria)

12

Properties of Partitions

- If the partitions are not complete or disjoint, that means the partitions have not been considered carefully enough
- They should be reviewed carefully, like any design
- Different alternatives should be considered

13

Properties of Partitions

- We model the input domain in five steps …
  - Steps 1 and 2 move us from the implementation abstraction level to the design abstraction level (from chapter 2)
  - Steps 3 & 4 are entirely at the design abstraction level
  - Step 5 brings us back down to the implementation abstraction level

14

Modeling the Input Domain

- Step 1 : Identify testable functions
  - Individual methods have one testable function
  - Methods in a class often have the same characteristics
  - Programs have more complicated characteristics—modeling documents such as UML can be used to design characteristics
  - Systems of integrated hardware and software components can use devices, operating systems, hardware platforms, browsers, etc.

15

Modeling the Input Domain

- Step 2: Find all the parameters
  - Often fairly straightforward, even mechanical
  - Important to be complete
  - Methods: Parameters and state (non-local) variables used
  - Components: Parameters to methods and state variables
  - System: All inputs, including files and databases

16

Modeling the Input Domain (*cont.*)

- Step 3: Model the input domain
  - The domain is scoped by the parameters
  - The structure is defined in terms of characteristics
  - Each characteristic is partitioned into sets of blocks
  - Each block represents a set of values
  - This is the *most creative design step in using ISP*

17

Modeling the Input Domain

Step 3: Model input domain

- Partitioning characteristics into blocks and values is a very creative engineering step
- More blocks means more tests
- Partitioning often flows directly from the definition of characteristics and both steps are done together
  - Should evaluate them separately – sometimes fewer characteristics can be used with more blocks and vice versa

18

Modeling the Input Domain

Step 3: Model input domain

Strategies for identifying values :

- Include valid, invalid and special values
- Sub-partition some blocks
- Explore boundaries of domains
- Include values that represent "normal use"
- Try to balance the number of blocks in each characteristic
- Check for completeness and disjointness!

19

Modeling the Input Domain

- Step 4 : Apply a test criterion to choose combinations of values
  - A test input has a value for each parameter
  - One block for each characteristic
  - Choosing all combinations is usually infeasible
  - Coverage criteria allow subsets to be chosen

20

Modeling the Input Domain

- Step 5 Refine combinations of blocks into test inputs
  - Choose appropriate values from each block

21

Steps 1 & 2—Interface & Functionality-Based

```
public boolean findElement (List list, Object element)
// Effects: if list or element is null throw NullPointerException
//          else return true if element is in the list, false otherwise
```

Interface-Based Approach

Two parameters : list, element
Characteristics :
- list is null (block1 = true, block2 = false)
- list is empty (block1 = true, block2 = false)

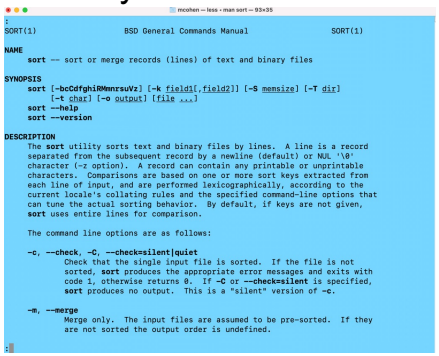
Functionality-Based Approach

Two parameters : list, element
Characteristics :
- number of occurrences of element in list
  (0, 1, >1)
- element occurs first in list
  (true, false)
- element occurs last in list
  (true, false)

22

In Class Exercise

Unix Sort Utility



```
SORT(1)                    BSD General Commands Manual                    SORT(1)

NAME
     sort -- sort or merge records (lines) of text and binary files

SYNOPSIS
     sort [-bcCdfghiRMmnrsuVz] [-k field1[,field2]] [-S memsize] [-T dir]
          [-t char] [-o output] [file ...]
     sort --help
     sort --version

DESCRIPTION
     The sort utility sorts text and binary files by lines.  A line is a record
     separated from the subsequent record by a newline (default) or NUL '\0'
     character (-z option).  A record can contain any printable or unprintable
     characters.  Comparisons are based on one or more sort keys extracted from
     each line of input, and are performed lexicographically, according to the
     current locale's collating rules and the specified command-line options that
     can tune the actual sorting behavior.  By default, if keys are not given,
     sort uses entire lines for comparison.

     The command line options are as follows:

     -c, --check, -C, --check=silent|quiet
             Check that the single input file is sorted.  If the file is not
             sorted, sort produces the appropriate error messages and exits with
             code 1, otherwise returns 0.  If -C or --check=silent is specified,
             sort produces no output.  This is a "silent" version of -c.

     -m, --merge
             Merge only.  The input files are assumed to be pre-sorted.  If they
             are not sorted the output order is undefined.
```

23

In Class Exercise

Unix Sort Utility

```
mcohen@atan-0202-02 ~ % sort --help
Usage: sort [-bcCdfigMmnrsuz] [-kPOS1[,POS2] ... ] [+POS1 [-POS2]] [-S memsize] [-T tmpdir] [
-t separator] [-o outfile] [--batch-size size] [--files0-from file] [--heapsort] [--mergesort
] [--radixsort] [--qsort] [--mmap] [--parallel thread_no] [--human-numeric-sort] [--version-s
ort] [--random-sort [--random-source file]] [--compress-program program] [file ...]
```

24

Using More than One IDM

- Some programs may have dozens or even hundreds of parameters
- Create several small IDMs
  - A divide-and-conquer approach
- Different parts of the software can be tested with different amounts of rigor
  - For example, some IDMs may include a lot of invalid values
- It is okay if the different IDMs overlap
  - The same variable may appear in more than one IDM

25

Two Approaches to Input Domain Modeling

1. Interface-based approach
   - Develops characteristics directly from individual input parameters
   - Simplest application
   - Can be partially automated in some situations

26

Two Approaches to Input Domain Modeling

2. Functionality-based approach
   - Develops characteristics from a behavioral view of the program under test
   - Harder to develop—requires more design effort
   - May result in better tests, or fewer tests that are as effective

Input Domain Model (IDM)

27

1. Interface-Based Approach

- Mechanically consider each parameter in isolation
- This is an easy modeling technique and relies mostly on syntax
- Some domain and semantic information won't be used
  - Could lead to an incomplete IDM
- Ignores relationships among parameters

28

1. Interface-Based Example

- Consider method *triang()* from class *TriangleType* on the book website :

```
public enum Triangle { Scalene, Isosceles, Equilateral, Invalid }
public static Triangle triang (int Side, int Side2, int Side3)
// Side1, Side2, and Side3 represent the lengths of the sides of a triangle
// Returns the appropriate enum value
```

The IDM for each parameter is identical

Reasonable characteristic : *Relation of side with zero*

29

2. Functionality-Based Approach

- Identify characteristics that correspond to the intended functionality
- Requires more design effort from tester
- Can incorporate domain and semantic knowledge
- Can use relationships among parameters

30

2. Functionality-Based Approach

- Modeling can be based on requirements, not implementation
- The same parameter may appear in multiple characteristics, so it's harder to translate values to test cases

31

2. Functionality-Based Example

- Ag… cla…

The three parameters represent a *triangle*

The IDM can combine all parameters

Reasonable characteristic : *Type of triangle*

32

Interface-Based –*triang()*

- *triang*() has one testable function and three inte

First Characterization of TriTyp's Inputs

Characteristic	b_1	b_2	b_3
q_1 = "Relation of Side 1 to 0"	greater than 0	equal to 0	less than 0
q_2 = "Relation of Side 2 to 0"	greater than 0	equal to 0	less than 0
q_3 = "Relation of Side 3 to 0"	greater than 0	equal to 0	less than 0

- A maximum of 3*3*3 = 27 tests
- Some triangles are valid, some are invalid
- Refining the characterization can lead to more tests …

33

Second Characterization of triang()'s Inputs

Characteristic	b_1	b_2	b_3	b_4
q_1 = "Refinement of q_1"	greater than 1	equal to 1	equal to 0	less than 0
q_2 = "Refinement of q_2"	greater than 1	equal to 1	equal to 0	less than 0
q_3 = "Refinement of q_3"	greater than 1	equal to 1	equal to 0	less than 0

- A maximum of 4*4*4 = 64 tests
- Complete because the inputs are integers (0 . . 1)

Possible values for partition q_1

Characteristic	b_1	b_2	b_3	b_4
Side1	2	1	0	-1

Test boundary conditions

34

Boundary Conditions

- We always need to consider the "boundaries" of a problem
- If we are testing something that cannot be zero, we need to test 0, 1, -1
- If we are searching for something in a using a list, we want to search the first, second, last, next to last position
- We place "boundaries" around any significant requirement

35

Functionality-Based *IDM—triang*()

- First two characterizations are based on syntax–parameters and their type
- A semantic level characterization could use the fact that the three

Geometric Characterization of *triang*()'s Inputs

Characteristic	b_1	b_2	b_3	b_4
q_1 = "Geometric Classification"	scalene	isosceles	equilateral	invalid

- Oops … something's fishy … equilateral is also isosceles !
- We need to refine the example to make characteristics valid

Correct Geometric Characterization of *triang*()'s Inputs

Characteristic	b_1	b_2	b_3	b_4
q_1 = "Geometric Classification"	scalene	isosceles, not equilateral	equilateral	invalid

36

Functionality-Based IDM—*triang*()

- Values for this partitioning can be chosen as

Possible values for geometric partition q_1				
Characteristic	b_1	b_2	b_3	b_4
Triangle	(4, 5, 6)	(3, 3, 4)	(3, 3, 3)	(3, 4, 8)

Functionality-Based IDM—*triang*()

- A different approach would be to break the geometric characterization into four separate characteristics

Four Characteristics for *triang*()		
Characteristic	b_1	b_2
q_1 = "Scalene"	True	False
q_2 = "Isosceles"	True	False
q_3 = "Equilateral"	True	False
q_4 = "Valid"	True	False

- Use constraints to ensure that
  - Equilateral = True implies Isosceles = True
  - Valid = False implies Scalene = Isosceles = Equilateral = False